



LOAD (INTEGRATE) DATA

```
In [1]: # Import libraries
import pandas as pd
import numpy as np
import joblib
from sklearn.preprocessing import RobustScaler, LabelEncoder
import matplotlib.pyplot as plt
import seaborn as sns

import os
```

```
In [2]: # Read data from csv files
data_path = 'dataset/init' # Trỏ đường dẫn vào thư mục con chứa file

weather = pd.read_csv(os.path.join(data_path, 'weather.csv'))
```

OVERVIEW

```
In [3]: weather.shape
```

```
Out[3]: (181960, 10)
```

```
In [4]: weather.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 181960 entries, 0 to 181959
Data columns (total 10 columns):
#   Column      Non-Null Count  Dtype
---  ---
0   province    181960 non-null object
1   max         181960 non-null int64
2   min         181960 non-null int64
3   wind        181960 non-null int64
4   wind_d      181960 non-null object
5   rain        181960 non-null float64
6   humidity    181960 non-null int64
7   cloud       181960 non-null int64
8   pressure    181960 non-null int64
9   date        181960 non-null object
dtypes: float64(1), int64(6), object(3)
memory usage: 13.9+ MB
```

Meaning of the columns

1. province: Province (or city) where data recorded
2. max: Maximum daily temperature (Celsius)

3. min: Minimum daily temperature (Celsius)
4. wind: Wind speed (km/h)
5. wind_d: Wind direction
6. rain: Amount of rain (mm)
7. humidi: Humidity (%)
8. cloud: Cloud coverage (%)
9. pressure: Air pressure (giá trị quan sát cho thấy nhiều khả năng là hPa/mbar)
10. date: Date record (yyyy-mm-dd)

In [5]: `weather.head()`

Out[5]:

	province	max	min	wind	wind_d	rain	humidi	cloud	pressure	date
0	Bac Lieu	27	22	17	NNE	6.9	90	71	1010	2009-01-01
1	Bac Lieu	31	25	20	ENE	0.0	64	24	1010	2010-01-01
2	Bac Lieu	29	24	14	E	0.0	75	45	1008	2011-01-01
3	Bac Lieu	30	24	30	E	0.0	79	52	1012	2012-01-01
4	Bac Lieu	31	25	20	ENE	0.0	70	24	1010	2013-01-01

In [6]: `print(weather.isnull().sum())`

```

province    0
max         0
min         0
wind        0
wind_d      0
rain        0
humidi      0
cloud       0
pressure    0
date        0
dtype: int64

```

In [7]: `print("\nNumber of duplicate rows:", weather.duplicated().sum())`

Number of duplicate rows: 0

CLEAN DATA

Equate the two values: Ha Noi and HaNoi in the Province column

```
In [8]: weather['province'] = weather['province'].replace('Hanoi', 'Ha Noi')
print(f"Total number of provinces/cities after merger: {weather['province'].nu
```

Total number of provinces/cities after merger: 39

Convert the date column to date and time format and set it as the index

```
In [9]: weather['date'] = pd.to_datetime(weather['date'])
weather.set_index('date', inplace=True)
print(f"Date range: {weather.index.min()} to {weather.index.max()}")
```

Date range: 2009-01-01 00:00:00 to 2021-06-18 00:00:00

Check for missing dates

```
In [10]: # Ensure the time series is not broken for each province
missing_dates = []
for prov in weather['province'].unique():
    prov_data = weather[weather['province'] == prov]
    # Create a standard date range from min to max
    expected_dates = pd.date_range(start=prov_data.index.min(), end=prov_data.index.max())
    # Find missing dates
    missing = expected_dates.difference(prov_data.index)
    if len(missing) > 0:
        missing_dates.append((prov, len(missing)))

if missing_dates:
    print("Provinces with missing dates:", missing_dates)
else:
    print("No missing dates in the data.")
```

Provinces with missing dates: [('Bac Lieu', 3), ('Ho Chi Minh City', 3), ('Tam Ky', 3), ('Ben Tre', 3), ('Hoa Binh', 3), ('Tan An', 3), ('Bien Hoa', 3), ('Hong Gai', 3), ('Thai Nguyen', 3), ('Buon Me Thuot', 3), ('Hue', 3), ('Thanh Hoa', 3), ('Ca Mau', 3), ('Long Xuyen', 3), ('Tra Vinh', 3), ('Cam Pha', 3), ('My Tho', 3), ('Tuy Hoa', 3), ('Cam Ranh', 3), ('Nam Dinh', 3), ('Uong Bi', 3), ('Can Tho', 3), ('Nha Trang', 3), ('Viet Tri', 3), ('Chau Doc', 3), ('Phan Rang', 3), ('Vinh', 3), ('Da Lat', 3), ('Phan Thiet', 3), ('Vinh Long', 3), ('Ha Noi', 3), ('Play Cu', 3), ('Vung Tau', 3), ('Hai Duong', 3), ('Qui Nhon', 3), ('Yen Bai', 3), ('Hai Phong', 3), ('Rach Gia', 3), ('Soc Trang', 3)]

Processing missing dates (Imputation)

```
In [11]: filled_data = []

for prov in weather['province'].unique():
    # Get data for each province.
    prov_data = weather[weather['province'] == prov].copy()
    prov_data = prov_data.sort_index()

    # Remove duplicate dates before reindexing
    if prov_data.index.duplicated().any():
        prov_data = prov_data.groupby(level=0).agg({
            'province': 'first',
            'max': 'mean',
            'min': 'mean',
            'wind': 'mean',
            'wind_d': 'first',
            'rain': 'mean',
            'humidi': 'mean',
            'cloud': 'mean',
            'pressure': 'mean'
        })

    # Create a standard date range from min to max
    full_date_range = pd.date_range(start=prov_data.index.min(), end=prov_data.index.max())

    # Update the Index (missing days will automatically generate new rows with NaN)
    prov_data = prov_data.reindex(full_date_range)

    # 1. Fill in the province name for the new rows with NaN
    prov_data['province'] = prov

    # 2. Interpolation for weather data columns (numeric)
    num_cols = ['max', 'min', 'wind', 'rain', 'humidi', 'cloud', 'pressure']
    prov_data[num_cols] = prov_data[num_cols].interpolate(method='time') # Interpolate

    # 3. Fill missing values for wind direction (string) with the previous day
    prov_data['wind_d'] = prov_data['wind_d'].ffill().bfill()

    # Restore integer columns after interpolation
    int_cols = ['max', 'min', 'wind', 'humidi', 'cloud', 'pressure']
    prov_data[int_cols] = prov_data[int_cols].round().astype(int)

    # Append to the list for concatenation
    filled_data.append(prov_data)

# Concatenate all province data into a complete DataFrame
weather = pd.concat(filled_data)
weather.index.name = 'date' # Rename the Index column
weather = weather.sort_index()

# Check if there are any missing dates left
missing_after = weather.isnull().sum().sum()
```

```
print(f"Data filling complete! Remaining NaN values: {missing_after}")
print(f"DataFrame size after adding missing dates: {weather.shape}")
```

Data filling complete! Remaining NaN values: 0
DataFrame size after adding missing dates: (177528, 9)

Rename the two columns: max, min

```
In [12]: weather = weather.rename(columns={
        'max': 'max_temp',
        'min': 'min_temp'
    })
print(weather.columns)
```

```
Index(['province', 'max_temp', 'min_temp', 'wind', 'wind_d', 'rain', 'humidi',
      'cloud', 'pressure'],
      dtype='object')
```

Add a column describing the temperature difference.

```
In [13]: weather['temp_diff'] = weather['max_temp'] - weather['min_temp']

print('Temperature difference column created: temp_diff')
print(weather[['max_temp', 'min_temp', 'temp_diff']].head())
```

```
Temperature difference column created: temp_diff
      max_temp  min_temp  temp_diff
date
2009-01-01      27      22         5
2009-01-01      24      23         1
2009-01-01      24      24         0
2009-01-01      22      18         4
2009-01-01      26      21         5
```

Check for outliers

Statistical Summary

```
In [14]: print("\nStatistical summary:")
print(weather.describe())
```

Statistical summary:

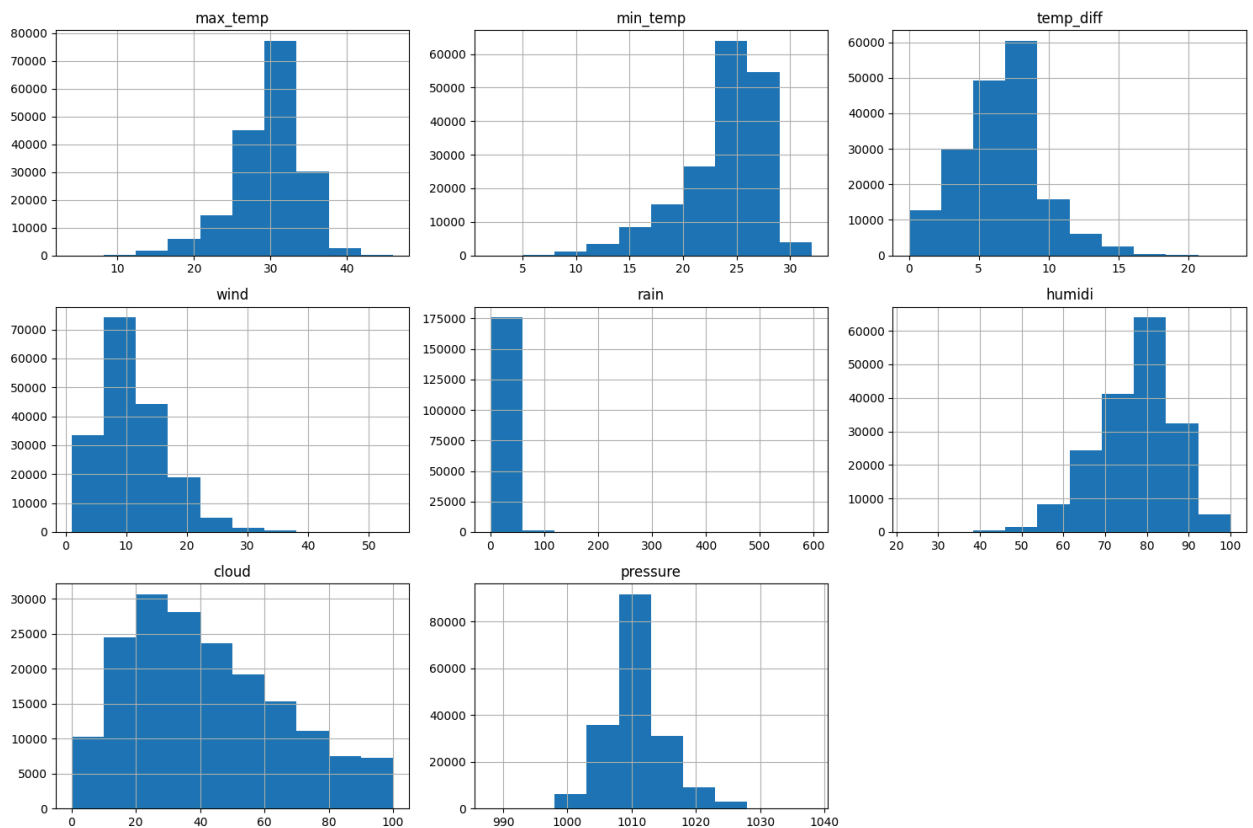
	max_temp	min_temp	wind	rain \
count	177528.000000	177528.000000	177528.000000	177528.000000
mean	29.845281	23.316057	11.108062	6.624552
std	4.521495	3.909861	5.338589	13.691810
min	4.000000	2.000000	1.000000	0.000000
25%	28.000000	22.000000	7.000000	0.100000
50%	31.000000	24.000000	10.000000	1.800000
75%	33.000000	26.000000	14.000000	7.600000
max	46.000000	32.000000	54.000000	596.400000

	humidi	cloud	pressure	temp_diff
count	177528.000000	177528.000000	177528.000000	177528.000000
mean	77.152393	41.494609	1010.205928	6.529224
std	9.234769	23.771185	4.560760	2.873500
min	23.000000	0.000000	988.000000	0.000000
25%	72.000000	23.000000	1008.000000	5.000000
50%	78.000000	38.000000	1010.000000	6.000000
75%	84.000000	58.000000	1012.000000	8.000000
max	100.000000	100.000000	1038.000000	23.000000

Distribution Visualization

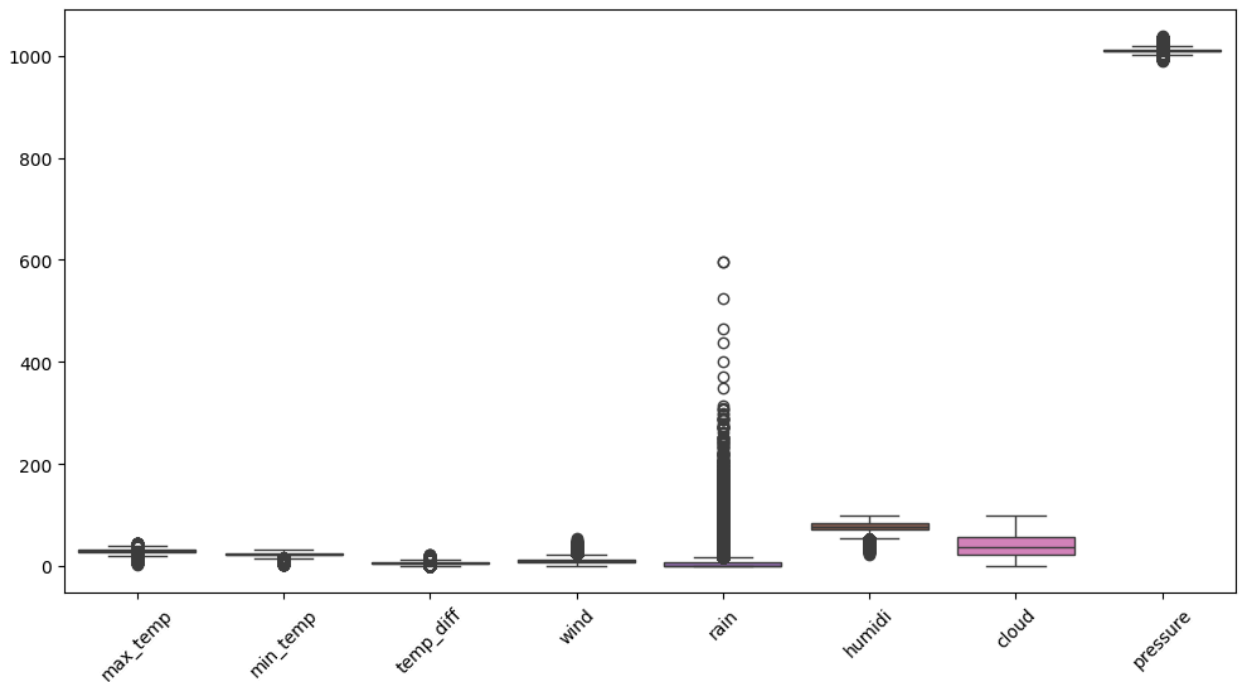
```
In [15]: num_cols = ['max_temp', 'min_temp', 'temp_diff', 'wind', 'rain', 'humidi', 'cl']

weather[num_cols].hist(figsize=(15, 10))
plt.tight_layout()
plt.show()
```



Boxplot (Outlier Detection)

```
In [16]: plt.figure(figsize=(12,6))
sns.boxplot(data=weather[num_cols])
plt.xticks(rotation=45)
plt.show()
```



Outlier Detection (IQR Method)

```
In [17]: def detect_outliers(df, col):
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    return df[(df[col] < lower) | (df[col] > upper)]

# Report outlier counts for all numeric columns (informational only – no data
num_cols = ['max_temp', 'min_temp', 'temp_diff', 'wind', 'rain', 'humidi', 'cl
print("Outlier counts per column (IQR method – kept in dataset):")
for col in num_cols:
    n = len(detect_outliers(weather, col))
    print(f"    {col}: {n} ({n / len(weather) * 100:.2f}%)")

print("\nNOTE: All flagged values are retained – they represent real extreme w
```

Outlier counts per column (IQR method – kept in dataset):

```
max_temp: 8087 (4.56%)
min_temp: 9751 (5.49%)
temp_diff: 5633 (3.17%)
wind: 4218 (2.38%)
rain: 16638 (9.37%)
humidi: 2214 (1.25%)
cloud: 0 (0.00%)
pressure: 13541 (7.63%)
```

NOTE: All flagged values are retained – they represent real extreme weather signals.

Data Validation (Business Rules)

```
In [18]: def validate_data(df):
          errors = []

          if not ((df['max_temp'] >= -10) & (df['max_temp'] <= 50)).all():
              errors.append("Max temperature out of range")

          if not ((df['min_temp'] >= -10) & (df['min_temp'] <= 50)).all():
              errors.append("Min temperature out of range")

          if not (df['temp_diff'] >= 0).all():
              errors.append("Temperature difference negative values found")

          if not ((df['humidi'] >= 0) & (df['humidi'] <= 100)).all():
              errors.append("Humidity out of range")

          if not ((df['cloud'] >= 0) & (df['cloud'] <= 100)).all():
              errors.append("Cloud out of range")

          if not (df['rain'] >= 0).all():
              errors.append("Rain negative values found")

          if not (df['pressure'] > 900).all():
              errors.append("Pressure out of range")

          return errors

errors = validate_data(weather)

if errors:
    print("Errors found:", errors)
else:
    print("All data within valid ranges")
```

All data within valid ranges

Comments on Outliers in Weather Data

The Nature of Outliers in Weather Data:

- Points marked by IQR — especially in the `rain` column — are not noise or system errors. They accurately reflect extreme weather events such as storms, tropical depressions, or record-breaking heavy rainfall.

Related to Model Objectives:

- One of the expected outputs is predicting extreme weather (wind gusts, heavy rainfall). If we remove or clip these values, we inadvertently remove the most important **signals** that XGBoost and LSTM need to learn to issue warnings.

Decision on handling:

- **Keep all outlier values unchanged.** Normalization using `RobustScaler` (which uses median and IQR) is robust to outliers, reducing their influence on gradient-based models without removing them.

TRANSFORM DATA

Cyclical time encoding (sin/cos)

```
In [19]: # Cyclical encoding for month and day-of-year
day_of_year = weather.index.dayofyear
weather['month_sin'] = np.sin(2 * np.pi * weather.index.month / 12)
weather['month_cos'] = np.cos(2 * np.pi * weather.index.month / 12)
weather['day_sin'] = np.sin(2 * np.pi * day_of_year / 365)
weather['day_cos'] = np.cos(2 * np.pi * day_of_year / 365)

print("Cyclical features created: month_sin, month_cos, day_sin, day_cos")
```

Cyclical features created: month_sin, month_cos, day_sin, day_cos

Group the provinces/cities according to 4 main climate zones and assign labels

```
In [20]: region_mapping = {
    'Bac Bo': ['Hoa Binh', 'Hong Gai', 'Thai Nguyen', 'Cam Pha', 'Nam Dinh', ' '
    'Trung Bo': ['Tam Ky', 'Hue', 'Thanh Hoa', 'Tuy Hoa', 'Cam Ranh', 'Nha Tra
    'Tay Nguyen': ['Buon Me Thuot', 'Da Lat', 'Play Cu'],
```

```
'Nam Bo': ['Bac Lieu', 'Ho Chi Minh City', 'Ben Tre', 'Tan An', 'Bien Hoa']
}
```

Create a reverse dictionary to map each province to a region

```
In [21]: prov_to_region = {prov: region for region, provs in region_mapping.items() for
```

Add a region column

```
In [22]: weather['region'] = weather['province'].map(prov_to_region)
```

Encode the region into numbers (0, 1, 2, 3) to feed into the Machine Learning model

```
In [23]: le_region = LabelEncoder()
weather['region_encoded'] = le_region.fit_transform(weather['region'])
```

```
In [24]: le_province = LabelEncoder()
weather['province_encoded'] = le_province.fit_transform(weather['province'])
```

Wind direction encoding (compass degrees → sin/cos)

```
In [25]: # Wind direction → compass degrees → sin/cos
wind_dir_degrees = {
    "N": 0, "NNE": 22.5, "NE": 45, "ENE": 67.5,
    "E": 90, "ESE": 112.5, "SE": 135, "SSE": 157.5,
    "S": 180, "SSW": 202.5, "SW": 225, "WSW": 247.5,
    "W": 270, "WNW": 292.5, "NW": 315, "NNW": 337.5,
}

weather['wind_deg'] = weather['wind_d'].map(wind_dir_degrees)
weather['wind_dir_sin'] = np.sin(np.deg2rad(weather['wind_deg']))
weather['wind_dir_cos'] = np.cos(np.deg2rad(weather['wind_deg']))
weather = weather.drop(columns=['wind_deg'])

print("Wind direction cyclical features created: wind_dir_sin, wind_dir_cos")
```

Wind direction cyclical features created: wind_dir_sin, wind_dir_cos

Export province-region mapping, remove text columns, and sort

```
In [26]: # Province-region mapping export
```

```

province_region_mapping = (
    weather[['province', 'region', 'province_encoded', 'region_encoded']]
    .drop_duplicates()
    .sort_values(['region_encoded', 'province_encoded'])
    .reset_index(drop=True)
)
province_region_mapping.to_csv('dataset/province_region_code_mapping.csv', index=False)
print("Province-region-code mapping exported: dataset/province_region_code_mapping.csv")

# Remove text columns no longer needed for modeling
weather_transformed = weather.drop(columns=['province', 'wind_d', 'region'])

# Reset index to make date a column, then sort by province_encoded and date
weather_transformed = weather_transformed.reset_index()
weather_transformed = weather_transformed.sort_values(['province_encoded', 'date'])

print(f"Data after initial transformation: {weather_transformed.shape}")
display(weather_transformed.head())

```

Province-region-code mapping exported: dataset/province_region_code_mapping.csv
Data after initial transformation: (177528, 17)

	date	max_temp	min_temp	wind	rain	humidi	cloud	pressure	temp_d
0	2009-01-01	27	22	17	6.9	90	71	1010	
1	2009-01-02	28	22	15	0.5	85	61	1010	
2	2009-01-03	23	21	9	16.7	91	77	1011	
3	2009-01-04	27	21	9	2.2	86	32	1011	
4	2009-01-05	29	22	10	0.0	81	25	1010	

Feature Engineering

Create domain-knowledge features, lag features, rolling statistics, and transition indicators per province. All lag/rolling features use `shift(1)` to prevent data leakage.

```

In [27]: # — Rainy season flag (domain knowledge per region) —————
rainy_months = {
    "Bac Bo": [5, 6, 7, 8, 9, 10],
    "Trung Bo": [9, 10, 11, 12],
    "Tay Nguyen": [5, 6, 7, 8, 9, 10],
    "Nam Bo": [5, 6, 7, 8, 9, 10, 11],
}

region_to_code = dict(zip(province_region_mapping['region'], province_region_mapping['region_encoded']))
rainy_set = set()
for region, months in rainy_months.items():
    code = region_to_code[region]
    for m in months:

```

```

        rainy_set.add((code, m))

month_col = pd.to_datetime(weather_transformed['date']).dt.month
weather_transformed['is_rainy_season'] = [
    1 if (rc, m) in rainy_set else 0
    for rc, m in zip(weather_transformed['region_encoded'], month_col)
]
print("+ is_rainy_season")

# — Humidity × cloud interaction —
weather_transformed['humid_cloud'] = weather_transformed['humidi'] * weather_t
print("+ humid_cloud")

# — Rain lag features (per province) —
for lag in [1, 2, 3, 5]:
    col_name = 'rain_yesterday' if lag == 1 else f'rain_lag_{lag}d'
    weather_transformed[col_name] = (
        weather_transformed.groupby('province_encoded')['rain']
        .transform(lambda s: s.shift(lag))
    )
    weather_transformed[col_name] = weather_transformed[col_name].fillna(0)
print("+ rain_yesterday, rain_lag_2d, rain_lag_3d, rain_lag_5d")

# — Rolling rain features (per province, shift=1 to avoid leakage) —
for w in [3, 7, 14, 30]:
    weather_transformed[f'rain_roll_mean_{w}d'] = (
        weather_transformed.groupby('province_encoded')['rain']
        .transform(lambda s: s.shift(1).rolling(w, min_periods=1).mean())
    )
    weather_transformed[f'rain_roll_max_{w}d'] = (
        weather_transformed.groupby('province_encoded')['rain']
        .transform(lambda s: s.shift(1).rolling(w, min_periods=1).max())
    )
    weather_transformed[f'rain_roll_std_{w}d'] = (
        weather_transformed.groupby('province_encoded')['rain']
        .transform(lambda s: s.shift(1).rolling(w, min_periods=1).std().fillna(0))
    )
print("+ rain_roll_mean/max/std for 3d, 7d, 14d, 30d")

# — Exponentially weighted rain (per province, shift=1, adjust=False) —
for span in [7, 14]:
    weather_transformed[f'rain_ewm_{span}d'] = (
        weather_transformed.groupby('province_encoded')['rain']
        .transform(lambda s: s.shift(1).ewm(span=span, adjust=False, min_peric
    )
weather_transformed['rain_ewm_7d'] = weather_transformed['rain_ewm_7d'].fillna(0)
weather_transformed['rain_ewm_14d'] = weather_transformed['rain_ewm_14d'].fillna(0)
print("+ rain_ewm_7d, rain_ewm_14d")

# — Pressure diff (day-to-day per province) —
weather_transformed['pressure_diff'] = (
    weather_transformed.groupby('province_encoded')['pressure']
    .transform(lambda s: s.diff().fillna(0))

```

```

)
print("+ pressure_diff")

# — Consecutive dry days (rain < 0.1) per province —————
weather_transformed['consecutive_dry_days'] = 0
for prov in weather_transformed['province_encoded'].unique():
    mask = weather_transformed['province_encoded'] == prov
    dry = (weather_transformed.loc[mask, 'rain'] < 0.1).astype(int).values
    counts = np.zeros(len(dry), dtype=int)
    for i in range(1, len(dry)):
        counts[i] = (counts[i - 1] + 1) * dry[i]
    weather_transformed.loc[mask, 'consecutive_dry_days'] = counts
print("+ consecutive_dry_days")

# — Temperature-humidity interaction —————
weather_transformed['temp_humid_interaction'] = weather_transformed['temp_diff']
print("+ temp_humid_interaction")

# — Heavy rain yesterday (>=20mm) —————
weather_transformed['heavy_rain_yesterday'] = (weather_transformed['rain_yesterday'] >= 20)
print("+ heavy_rain_yesterday")

# — Wet/dry transitions (rain >= 1.0 = wet) —————
is_wet_today = (weather_transformed['rain'] >= 1.0).astype(int)
is_wet_yesterday = (weather_transformed['rain_yesterday'] >= 1.0).astype(int)
weather_transformed['dry_to_wet'] = ((is_wet_yesterday == 0) & (is_wet_today == 1))
weather_transformed['wet_to_dry'] = ((is_wet_yesterday == 1) & (is_wet_today == 0))

# Wet streak days (same algorithm as consecutive_dry_days but for wet>=1.0)
weather_transformed['wet_streak_days'] = 0
for prov in weather_transformed['province_encoded'].unique():
    mask = weather_transformed['province_encoded'] == prov
    wet = is_wet_today[mask].values
    counts = np.zeros(len(wet), dtype=int)
    for i in range(1, len(wet)):
        counts[i] = (counts[i - 1] + 1) * wet[i]
    weather_transformed.loc[mask, 'wet_streak_days'] = counts
print("+ dry_to_wet, wet_to_dry, wet_streak_days")

# — Pressure & wind rolling means (per province, shift=1) —————
weather_transformed['pressure_roll_mean_3d'] = (
    weather_transformed.groupby('province_encoded')['pressure']
    .transform(lambda s: s.shift(1).rolling(3, min_periods=1).mean())
)
weather_transformed['wind_roll_mean_3d'] = (
    weather_transformed.groupby('province_encoded')['wind']
    .transform(lambda s: s.shift(1).rolling(3, min_periods=1).mean())
)
weather_transformed['pressure_roll_mean_3d'] = weather_transformed['pressure_roll_mean_3d']
weather_transformed['wind_roll_mean_3d'] = weather_transformed['wind_roll_mean_3d']
print("+ pressure_roll_mean_3d, wind_roll_mean_3d")

# — Log-transform target —————

```

```

weather_transformed['rain_loglp'] = np.loglp(weather_transformed['rain'])
print("+ rain_loglp")

# — Fill any remaining NaN from lag/diff features —
lag_cols = [c for c in weather_transformed.columns if 'roll_' in c or 'ewm_' in c]
weather_transformed[lag_cols] = weather_transformed[lag_cols].fillna(0)

print(f"\nFeature engineering complete. Shape: {weather_transformed.shape}")

+ is_rainy_season
+ humid_cloud
+ rain_yesterday, rain_lag_2d, rain_lag_3d, rain_lag_5d
+ rain_roll_mean/max/std for 3d, 7d, 14d, 30d
+ rain_ewm_7d, rain_ewm_14d
+ pressure_diff
+ consecutive_dry_days
+ temp_humid_interaction
+ heavy_rain_yesterday
+ dry_to_wet, wet_to_dry, wet_streak_days
+ pressure_roll_mean_3d, wind_roll_mean_3d
+ rain_loglp

```

Feature engineering complete. Shape: (177528, 47)

Select and order output columns

```

In [28]: # Define final output column order (must match dataset_check format)
OUTPUT_COLS = [
    'date', 'max_temp', 'min_temp', 'wind', 'humidi', 'cloud', 'pressure', 'te
    'month_sin', 'month_cos', 'day_sin', 'day_cos', 'wind_dir_sin', 'wind_dir_
    'is_rainy_season', 'humid_cloud', 'rain_yesterday',
    'rain_roll_mean_7d', 'rain_roll_max_7d', 'rain_roll_std_7d',
    'rain_roll_mean_14d', 'rain_roll_max_14d', 'rain_roll_std_14d',
    'rain_roll_mean_30d', 'rain_roll_max_30d', 'rain_roll_std_30d',
    'pressure_diff', 'consecutive_dry_days', 'temp_humid_interaction',
    'region_encoded', 'province_encoded',
    'rain_lag_2d', 'rain_lag_3d', 'rain_lag_5d',
    'rain_roll_mean_3d', 'rain_roll_max_3d', 'rain_roll_std_3d',
    'rain_ewm_7d', 'rain_ewm_14d',
    'heavy_rain_yesterday', 'dry_to_wet', 'wet_to_dry', 'wet_streak_days',
    'pressure_roll_mean_3d', 'wind_roll_mean_3d',
    'rain_loglp', 'rain',
]

weather_final = weather_transformed[OUTPUT_COLS].copy()
print(f"Final dataset shape: {weather_final.shape}")
print(f"Columns ({len(weather_final.columns)}):")
print(list(weather_final.columns))

```

Final dataset shape: (177528, 47)

Columns (47):

```
['date', 'max_temp', 'min_temp', 'wind', 'humid', 'cloud', 'pressure', 'temp_diff', 'month_sin', 'month_cos', 'day_sin', 'day_cos', 'wind_dir_sin', 'wind_dir_cos', 'is_rainy_season', 'humid_cloud', 'rain_yesterday', 'rain_roll_mean_7d', 'rain_roll_max_7d', 'rain_roll_std_7d', 'rain_roll_mean_14d', 'rain_roll_max_14d', 'rain_roll_std_14d', 'rain_roll_mean_30d', 'rain_roll_max_30d', 'rain_roll_std_30d', 'pressure_diff', 'consecutive_dry_days', 'temp_humid_interaction', 'region_encoded', 'province_encoded', 'rain_lag_2d', 'rain_lag_3d', 'rain_lag_5d', 'rain_roll_mean_3d', 'rain_roll_max_3d', 'rain_roll_std_3d', 'rain_ewm_7d', 'rain_ewm_14d', 'heavy_rain_yesterday', 'dry_to_wet', 'wet_to_dry', 'wet_streak_days', 'pressure_roll_mean_3d', 'wind_roll_mean_3d', 'rain_loglp', 'rain']
```

TRAIN / VALIDATION / TEST SPLIT

Temporal split to avoid data leakage:

- **Train:** 2009-01-01 → 2018-12-31
- **Validation:** 2019-01-01 → 2020-12-31
- **Test:** 2021-01-01 → 2021-06-18

In [29]: *# 3-way temporal split*

```
train_data = weather_final[weather_final['date'] <= '2018-12-31'].copy()
val_data = weather_final[(weather_final['date'] >= '2019-01-01') & (weather_final['date'] <= '2020-12-31')].copy()
test_data = weather_final[weather_final['date'] >= '2021-01-01'].copy()

print(f"Train set: {train_data.shape} ({train_data['date'].min()} → {train_data['date'].max()})")
print(f"Val set: {val_data.shape} ({val_data['date'].min()} → {val_data['date'].max()})")
print(f"Test set: {test_data.shape} ({test_data['date'].min()} → {test_data['date'].max()})")

# Verify no overlap
assert train_data['date'].max() < val_data['date'].min(), "Data leakage: train/val overlap"
assert val_data['date'].max() < test_data['date'].min(), "Data leakage: val/test overlap"
print("\n✓ No data leakage between splits")
```

Train set: (142428, 47) (2009-01-01 00:00:00 → 2018-12-31 00:00:00)

Val set: (28509, 47) (2019-01-01 00:00:00 → 2020-12-31 00:00:00)

Test set: (6591, 47) (2021-01-01 00:00:00 → 2021-06-18 00:00:00)

✓ No data leakage between splits

NORMALIZE (RobustScaler) & EXPORT

- **RobustScaler** is used instead of MinMaxScaler — more robust to outliers.
- Feature scaler and target scaler are fit **only on training data** to prevent data leakage.

- Columns NOT scaled: `region_encoded`, `province_encoded`, `rain` (raw target), `date`.

```
In [30]: # Columns NOT to scale (categorical codes + raw target)
NO_SCALE_COLS = {'date', 'region_encoded', 'province_encoded', 'rain'}

# Feature columns to scale (everything except no-scale and the target rain_loglp
FEATURE_SCALE_COLS = [c for c in OUTPUT_COLS if c not in NO_SCALE_COLS and c != TARGET_COL]
TARGET_COL = 'rain_loglp'

# Fit RobustScaler on training data only
feat_scaler = RobustScaler()
feat_scaler.fit(train_data[FEATURE_SCALE_COLS])

tgt_scaler = RobustScaler()
tgt_scaler.fit(train_data[[TARGET_COL]])

# Apply scaling to all splits
train_scaled = train_data.copy()
val_scaled = val_data.copy()
test_scaled = test_data.copy()

for df_scaled, df_orig in [(train_scaled, train_data), (val_scaled, val_data),
                           (test_scaled, test_data)]:
    df_scaled[FEATURE_SCALE_COLS] = feat_scaler.transform(df_orig[FEATURE_SCALE_COLS])
    df_scaled[TARGET_COL] = tgt_scaler.transform(df_orig[[TARGET_COL]])

print(f"Feature scaler fitted on {len(FEATURE_SCALE_COLS)} columns")
print(f"Target scaler fitted on: {TARGET_COL}")
print(f"Unscaled columns: {sorted(NO_SCALE_COLS - {'date'})}")

# — Export unscaled CSVs —
train_data.to_csv('dataset/weather_train_2009_2018.csv', index=False)
val_data.to_csv('dataset/weather_val_2019_2020.csv', index=False)
test_data.to_csv('dataset/weather_test_2021.csv', index=False)
print("\nUnscaled CSVs exported.")

# — Export scaled CSVs —
train_scaled.to_csv('dataset/weather_train_2009_2018_scaled.csv', index=False)
val_scaled.to_csv('dataset/weather_val_2019_2020_scaled.csv', index=False)
test_scaled.to_csv('dataset/weather_test_2021_scaled.csv', index=False)
print("Scaled CSVs exported.")

# — Save scalers —
joblib.dump({'feature_scaler': feat_scaler, 'target_scaler': tgt_scaler}, 'dataset/scalers.pkl')
print("Scalers saved: dataset/scalers.pkl")
```

Feature scaler fitted on 42 columns
 Target scaler fitted on: rain_loglp
 Unscaled columns: ['province_encoded', 'rain', 'region_encoded']

Unscaled CSVs exported.
 Scaled CSVs exported.
 Scalers saved: dataset/scalers.pkl

Notes on Segmentation, Normalization, and Features

- **3-way temporal split** (train/val/test) maintains chronological order and avoids data leakage.
- **RobustScaler** is preferred over MinMaxScaler for weather data with natural outliers (storms, extreme rainfall). It centers on the median and scales by IQR, reducing the influence of extreme values.
- **Unscaled data** is suitable for tree-based models (XGBoost, Random Forest).
- **Scaled data** is required for LSTM and gradient-sensitive models.
- For **Prophet**, create separate tables per province with `ds`, `y`, and exogenous variables.
- All **lag features** use `shift(1)` or higher to prevent data leakage — no future information is used.
- **Feature engineering is performed on the full dataset before splitting** so that validation/test sets have valid lag features from the preceding period.

QA REVIEW CHECKS

Verification that the processed data matches expected outputs.

```
In [31]: # — QA: Compare with dataset_check files —————
check_dir = 'dataset_check'
output_dir = 'dataset'

check_files = {
    'train': ('weather_train_2009_2018.csv', 'weather_train_2009_2018_scaled.csv'),
    'val': ('weather_val_2019_2020.csv', 'weather_val_2019_2020_scaled.csv'),
    'test': ('weather_test_2021.csv', 'weather_test_2021_scaled.csv'),
}

all_pass = True
for split_name, (unscaled_fn, scaled_fn) in check_files.items():
    for fn in [unscaled_fn, scaled_fn]:
        check_path = os.path.join(check_dir, fn)
        out_path = os.path.join(output_dir, fn)
        if not os.path.exists(check_path):
            print(f"△ Check file not found: {check_path}")
            continue

        df_check = pd.read_csv(check_path)
        df_out = pd.read_csv(out_path)

        # Shape match
```

```

shape_ok = df_check.shape == df_out.shape
# Column match
col_ok = list(df_check.columns) == list(df_out.columns)
# Value match (within tolerance for floats)
val_ok = True
max_diff = 0.0
for col in df_check.columns:
    if col == 'date':
        if not (df_check[col].values == df_out[col].values).all():
            val_ok = False
    else:
        diff = (df_check[col] - df_out[col]).abs().max()
        max_diff = max(max_diff, diff)
        if diff > 1e-4:
            val_ok = False
            print(f" x {fn} - column '{col}' max diff = {diff:.6f}")

status = "✓" if (shape_ok and col_ok and val_ok) else "x"
if not shape_ok:
    print(f" x {fn} - shape mismatch: {df_out.shape} vs expected {df_")
if not col_ok:
    print(f" x {fn} - column mismatch")
if shape_ok and col_ok and val_ok:
    print(f" {status} {fn} - MATCH (max diff: {max_diff:.2e})")
else:
    all_pass = False

print(f"\n{'='*50}")
if all_pass:
    print("ALL FILES MATCH dataset_check ✓")
else:
    print("SOME FILES DO NOT MATCH - check above for details")

✓ weather_train_2009_2018.csv - MATCH (max diff: 0.00e+00)
✓ weather_train_2009_2018_scaled.csv - MATCH (max diff: 0.00e+00)
✓ weather_val_2019_2020.csv - MATCH (max diff: 0.00e+00)
✓ weather_val_2019_2020_scaled.csv - MATCH (max diff: 0.00e+00)
✓ weather_test_2021.csv - MATCH (max diff: 0.00e+00)
✓ weather_test_2021_scaled.csv - MATCH (max diff: 0.00e+00)

```

ALL FILES MATCH dataset_check ✓